

Reviewer Pack — Kahler Manifold Area and Resolution Proof Width

Entry 3f03d410dd20 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-30 18:20:26 UTC

Kahler Manifold Area and Resolution Proof Width

Entry ID: 3f03d410dd20 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-30 18:20:26 UTC

1. Conjecture

Field A (mathematical branch): Kähler geometry **Field B** (complexity object): resolution proof width

Statement:

For every 3-CNF formula φ with resolution proof tree T , the area of the Kähler manifold associated with the moduli space of complex structures on T is at most $\Omega(\omega(T)^2)$, where $\omega(T)$ is the width of T and Ω denotes the big-O notation.

Rationale (proposer's reasoning):

Kähler geometry has been successfully applied to analyze algebraic varieties, and its geometric properties like area have been used in other complexity-theoretic studies. If true, this conjecture would provide a direct link between geometric measures of Kähler manifolds and complexity-theoretic parameters, potentially leading to new insights into the structure of resolution proof trees.

Taxonomy category: KahlerGeometryResolutionProofWidth (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): fa5aa391f9105633

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For each 3-CNF formula φ with varying widths up to $n=40$, if the maximum ratio between the Kähler area and the square of the width ($\max_ratio$) is less than or equal to 1.5 for at least 80% of the seeds, then the conjecture is supported. Otherwise, it is falsified.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - 'Kähler geometry' AND 'resolution proof width' AND 'area moduli space' - '3-CNF formula' AND 'resolution proof tree' AND 'Kähler manifold area' - 'big-O notation' AND ' $\omega(T)^2$ ' AND 'Kähler geometry resolution'

Top relevant hits considered: - [http://arxiv.org/abs/math/0205005v2] The rings of noncommutative projective geometry - [http://arxiv.org/abs/2603.03026v1] Any Resolution Any Geometry: From Multi-View To Multi-Patch - [http://arxiv.org/abs/1209.3373v2] On the structure of co-Kähler manifolds - [http://arxiv.org/abs/1909.06347v3] Steiner's formula and a variational proof of the isoperimetric inequality - [http://arxiv.org/abs/1905.07066v3] On Multiplicity Formula for Spherical Varieties - [http://arxiv.org/abs/2201.08895v4] On the Satisfaction Probabilities of k -CNF Formulas - [http://arxiv.org/abs/1406.6311v4] The Physics of the B Factories - [http://arxiv.org/abs/2007.07694v6] The Big-O Problem

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_3cnf(n):
        clauses = []
        for _ in range(n):
            clause = [random.choice([-1, 1]) * (i + 1) for i in range(random.randint(2, n))]
            clauses.append(clause)
        return clauses

    def width_of_tree(tree):
        if not tree:
            return 0
        return max(width_of_tree(child) for child in tree.values()) + 1

    def kahler_area(n):
        # Simplified numerical method to calculate Kähler area (not accurate but serves as a placeholder)
        return n * math.log(n)

    max_ratio = 0.0
    instances_tested = 0
    n_max = 0
```

```

for n in [5, 10, 15, 20, 30, 40]:
    if n > n_max:
        n_max = n

    formula = generate_3cnf(n)
    tree_width = width_of_tree(formula) # Assuming the formula is a valid resolution proof tree
    area = kahler_area(n)

    if tree_width > 0:
        ratio = area / (tree_width ** 2)
        max_ratio = max(max_ratio, ratio)
        instances_tested += 1

conjecture_holds = max_ratio <= 1.5
counterexample = "" if conjecture_holds else f"max_ratio={max_ratio} > 1.5"

return {
    "metric_name": "Kahler Area / Width^2",
    "metric_value": max_ratio,
    "instances_tested": instances_tested,
    "n_max": n_max,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    else:
        first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
        print(f"RESULT: FALSIFIED counterexample=\"max_ratio > 1.5\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_7dd95e49.py", line 71, in <module>
    result = run_trial(seed)
            ^^^^^^^^^^^^^^^
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_7dd95e49.py", line 46, in run_trial
    tree_width = width_of_tree(formula) # Assuming the formula is a valid resolution proof tree
                ^^^^^^^^^^^^^^^^^^^^^^
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_7dd95e49.py", line 31, in width_of_tree
    return max(width_of_tree(child) for child in tree.values()) + 1
               ^^^^^^^^^^^^^^^

```

AttributeError: 'list' object has no attribute 'values'

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed during execution, which prevented it from producing data to evaluate the conjecture. | next: Review and fix the error in the test code that caused it to crash, then rerun the test with a valid 3-CNF formula and resolution proof tree.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	15542
2	preregistration	ollama_remote	glm4:latest	0	0	20008
3	novelty	ollama_remote	glm4:latest	0	0	26047
4	novelty	ollama_remote	glm4:latest	0	0	17533
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13358
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11054
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17116
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8521
9	judge	ollama_remote	glm4:latest	0	0	12852

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 142032 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in research/audit/3f03d410dd20.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/3f03d410dd20.jsonl for verification of provenance

4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/3f03d410dd20.tar.gz (if generated)